

CS2089M

Database System

Lecture 14: Recovery System

Yunjun Gao (高云君)

College of Computer Science

Zhejiang University

Spring & Summer 2026

13957167510 / gaoyj@zju.edu.cn

Outline

- ❑ Failure Classification
- ❑ *Storage Structure
- ❑ *Recovery and Atomicity
- ❑ Log-Based Recovery
- ❑ *Shadow Paging
- ❑ Recovery With Concurrent Transactions
- ❑ Buffer Management
- ❑ Failure with Loss of Nonvolatile Storage
- ❑ *Advanced Recovery Techniques
- ❑ *ARIES Recovery Algorithm
- ❑ *Remote Backup Systems



Failure Classification

❑ **Transaction failure** :

- **Logical errors**: transaction cannot complete due to some internal error condition: **overflow**, **bad input**, **data not found**,...
- **System errors**: the database system must terminate an active transaction due to an error condition (e.g., **deadlock**)

❑ **System crash**: a **power failure** or other **hardware** or **software failure** causes the system to crash.

- **Fail-stop assumption**: non-volatile storage contents are assumed to not be corrupted by system crash
 - Database systems have numerous integrity checks to prevent corruption of disk data

❑ **Disk failure**: a head crash or similar disk failure **destroys all or part of disk storage**

- Destruction is assumed to be detectable: disk drives use checksums to detect failures

Recovery Algorithms

- ❑ Consider transaction T_i that transfers \$50 from account A to account B
 - Two updates: subtract 50 from A and add 50 to B
- ❑ Transaction T_i requires updates to A and B to be output to the database.
 - A failure may occur after one of these modifications have been made but before both of them are made.
 - Modifying the database without ensuring that the transaction will commit may leave the database in an inconsistent state
 - Not modifying the database may result in lost updates if failure occurs just after transaction commits
- ❑ Recovery algorithms have two parts
 1. Actions taken during normal transaction processing to ensure enough information exists to recover from failures
 2. Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability

Outline

- ☐ Failure Classification
- ☐ *Storage Structure
- ☐ *Recovery and Atomicity
- ☐ Log-Based Recovery
- ☐ *Shadow Paging
- ☐ Recovery With Concurrent Transactions
- ☐ Buffer Management
- ☐ Failure with Loss of Nonvolatile Storage
- ☐ *Advanced Recovery Techniques
- ☐ *ARIES Recovery Algorithm
- ☐ *Remote Backup Systems



Storage Structure

❑ **Volatile storage:**

- Does not survive system crashes
- Examples: main memory, cache memory

❑ **Nonvolatile storage:**

- Survives system crashes
- Examples: disk, tape, flash memory,
non-volatile (battery backed up) RAM
- But may still fail, losing data

❑ **Stable storage:**

- A mythical form of storage that survives all failures
- Approximated by maintaining multiple copies on distinct nonvolatile media
- See book for more details on how to implement stable storage

Stable-Storage Implementation

- ❑ Maintain multiple copies of each block on separate disks
 - Copies can be at remote sites to protect against disasters such as fire or flooding.
- ❑ Failure during data transfer can still result in inconsistent copies:
Block transfer can result in
 - Successful completion
 - Partial failure: destination block has incorrect information
 - Total failure: destination block was never updated
- ❑ Protecting storage media from failure during data transfer (one solution):
 - Execute output operation as follows (assuming two copies of each block):
 1. Write the information onto the first physical block.
 2. When the first write successfully completes, write the same information onto the second physical block.
 3. The output is completed only after the second write successfully completes.

Stable-Storage Implementation (Cont.)

- ❑ Protecting storage media from failure during data transfer (cont.):
- ❑ Copies of a block may differ due to failure during output operation.
To recover from failure:
 1. First find inconsistent blocks:
 1. *Expensive solution*: Compare the two copies of every disk block.
 2. *Better solution*:
 - Record in-progress disk writes on non-volatile storage (Non-volatile RAM or special area of disk).
 - Use this information during recovery to find blocks that may be inconsistent, and only compare copies of these.
 - Used in hardware RAID systems
 2. If either copy of an inconsistent block is detected to have an error (bad checksum), overwrite it by the other copy. If both have no error, but are different, overwrite the second block by the first block.

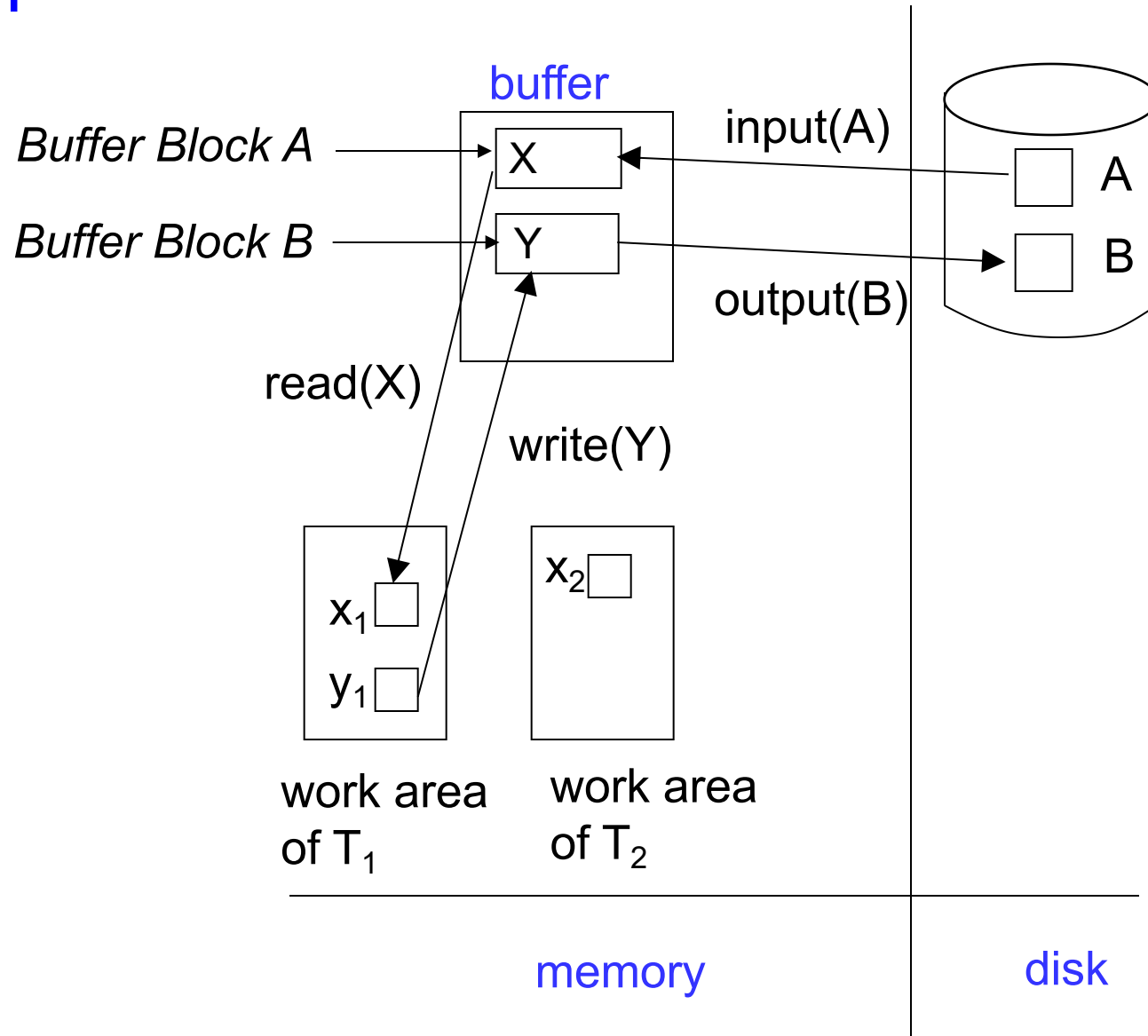
Data Access

- ❑ **Physical blocks** are those blocks residing on the disk.
- ❑ **Buffer blocks** are the blocks residing temporarily in main memory.
- ❑ Block movements between disk and main memory are initiated through the following two operations:
 - **input**(B) transfers the physical block B to main memory.
 - **output**(B) transfers the buffer block B to the disk, and replaces the appropriate physical block there.
- ❑ We assume, for simplicity, that each data item fits in, and is stored inside, a single block.

Data Access (Cont.)

- ❑ Each transaction T_i has its private work-area in which local copies of all data items accessed and updated by it are kept.
 - T_i 's local copy of a data item X is called x_i .
- ❑ Transferring data items between system buffer blocks and its private work-area done by:
 - **read**(X) assigns the value of data item X to the local variable x_i .
 - **write**(X) assigns the value of local variable x_i to data item $\{X\}$ in the buffer block.
 - **Note:** **output**(B_X) need not immediately follow **write**(X). System can perform the **output** operation when it deems fit.
- ❑ Transactions
 - Must perform **read**(X) before accessing X for the first time (subsequent reads can be from local copy)
 - **write**(X) can be executed at any time before the transaction commits

Example of Data Access



Outline

- ☐ Failure Classification
- ☐ *Storage Structure
- ☐ *Recovery and Atomicity
- ☐ Log-Based Recovery
- ☐ *Shadow Paging
- ☐ Recovery With Concurrent Transactions
- ☐ Buffer Management
- ☐ Failure with Loss of Nonvolatile Storage
- ☐ *Advanced Recovery Techniques
- ☐ *ARIES Recovery Algorithm
- ☐ *Remote Backup Systems



*Recovery and Atomicity

- ❑ To ensure atomicity despite failures, we first output information describing the modifications to stable storage without modifying the database itself.
- ❑ We study **log-based recovery mechanisms** in detail
 - We first present key concepts
 - And then present the actual recovery algorithm
- ❑ Less used alternative: **shadow-paging** (brief details in book)

Outline

- ☐ Failure Classification
- ☐ *Storage Structure
- ☐ *Recovery and Atomicity
- ☐ **Log-Based Recovery**
- ☐ *Shadow Paging
- ☐ Recovery With Concurrent Transactions
- ☐ Buffer Management
- ☐ Failure with Loss of Nonvolatile Storage
- ☐ *Advanced Recovery Techniques
- ☐ *ARIES Recovery Algorithm
- ☐ *Remote Backup Systems



Log-Based Recovery

- ❑ A **log** is kept on stable storage.
 - The log is a sequence of **log records**, and maintains a record of update activities on the database.
- ❑ When transaction T_i starts, it registers itself by writing a $\langle T_i \text{ start} \rangle$ log record
- ❑ Before T_i executes **write**(X), a log record $\langle T_i, X, V_1, V_2 \rangle$ is written, where V_1 is the value of X before the write (the **old value**), and V_2 is the value to be written to X (the **new value**).
- ❑ When T_i finishes its last statement, the log record $\langle T_i \text{ commit} \rangle$ is written.
- ❑ Two approaches using logs
 - Deferred database modification
 - Immediate database modification

Log File Example

- <T1 **start**>
- <T1, A, 100, 200>
- <T2 **start**>
- <T2, B, 300, 400>
- <T3 **start**>
- <T1, C, 500, 600>
- <T1 **commit**>
- <T3, C, 600, 700>
- <T3, C, 700, 800>
- <T3 **commit**>
- <T2, C, 800, 900>
- <T2, B, 400, 500>

Deferred Database Modification

- ❑ The **deferred database modification** scheme records all modifications to the log, but defers all the **writes** to after partial commit.
- ❑ Assume that transactions execute serially
- ❑ Transaction starts by writing $\langle T_i, \text{start} \rangle$ record to log.
- ❑ A **write**(X) operation results in a log record $\langle T_i, X, V \rangle$ being written, where V is the new value for X
 - Note: old value is not needed for this scheme
- ❑ The write is not performed on X at this time, but is deferred.
- ❑ When T_i partially commits, $\langle T_i, \text{commit} \rangle$ is written to the log
- ❑ Finally, the log records are read and used to actually execute the previously deferred writes.

Deferred Database Modification (Cont.)

□ Below we show the log as it appears at three instances of time.

$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$
$\langle T_0, A, 950 \rangle$	$\langle T_0, A, 950 \rangle$	$\langle T_0, A, 950 \rangle$
$\langle T_0, B, 2050 \rangle$	$\langle T_0, B, 2050 \rangle$	$\langle T_0, B, 2050 \rangle$
	$\langle T_0 \text{ commit} \rangle$	$\langle T_0 \text{ commit} \rangle$
	$\langle T_1 \text{ start} \rangle$	$\langle T_1 \text{ start} \rangle$
	$\langle T_1, C, 600 \rangle$	$\langle T_1, C, 600 \rangle$
		$\langle T_1 \text{ commit} \rangle$
(a)	(b)	(c)

□ If log on stable storage at time of crash is as in case:

- (a) No redo actions need to be taken
- (b) redo(T_0) must be performed since $\langle T_0 \text{ commit} \rangle$ is present
- (c) redo(T_0) must be performed followed by redo(T_1) since $\langle T_0 \text{ commit} \rangle$ and $\langle T_1 \text{ commit} \rangle$ are present

Immediate Database Modification

- ❑ The **immediate-modification** scheme allows updates of an uncommitted transaction to be made to the buffer, or the disk itself, before the transaction commits
- ❑ Update log record must be written *before* database item is written
 - We assume that the log record is output directly to stable storage
 - (Will see later that how to postpone log record output to some extent)
- ❑ Output of updated blocks to stable storage can take place at any time before or after transaction commit
- ❑ Order in which blocks are output can be different from the order in which they are written.

Immediate Database Modification Example

Log	Write	Output
$\langle T_0 \text{ start} \rangle$		
$\langle T_0, A, 1000, 950 \rangle$		
$\langle T_0, B, 2000, 2050 \rangle$		
	$A = 950$ $B = 2050$	
$\langle T_0 \text{ commit} \rangle$		
$\langle T_1 \text{ start} \rangle$		
$\langle T_1, C, 700, 600 \rangle$		
	$C = 600$	
$\langle T_1 \text{ commit} \rangle$		

Note: B_X denotes block containing X .

Immediate Database Modification (Cont.)

---How to recovery

- ❑ Recovery procedure has **two operations** instead of one:
 - **undo(T_i)** restores the value of all data items updated by T_i to their **old values, going backwards from the last log record** for T_i
 - **redo(T_i)** sets the value of all data items updated by T_i to the **new values, going forward from the first log record** for T_i
- ❑ Both operations must be idempotent (幂等)
 - That is, even if the operation is executed multiple times the effect is the same as if it is executed once
 - Needed since operations may get re-executed during recovery
- ❑ When recovering after failure:
 - Transaction T_i needs to be **undone** if the log **contains** the record **< T_i start>**, but does not contain the record **< T_i commit>**.
 - Transaction T_i needs to be **redone** if the log **contains both** the record **< T_i start>** and the record **< T_i commit>**.
- ❑ **Undo operations are performed first, then redo operations.**

Immediate DB Modification Recovery Example

Below we show the log as it appears at three instances of time.

< T_0 start>
< T_0 , A, 1000, 950>
< T_0 , B, 2000, 2050>

(a)

< T_0 start>
< T_0 , A, 1000, 950>
< T_0 , B, 2000, 2050>
< T_0 commit>
< T_1 start>
< T_1 , C, 700, 600>

(b)

< T_0 start>
< T_0 , A, 1000, 950>
< T_0 , B, 2000, 2050>
< T_0 commit>
< T_1 start>
< T_1 , C, 700, 600>
< T_1 commit>

(c)

Recovery actions in each case above are:

(a) undo (T_0): B is restored to 2000 and A to 1000, and log records
< T_0 , B, 2000>, < T_0 , A, 1000>, < T_0 , **abort**> are written out

(b) redo (T_0) and undo (T_1): A and B are set to 950 and 2050, and C is
restored to 700. Log records < T_1 , C, 700>, < T_1 , **abort**> are written out.

(c) redo (T_0) and redo (T_1): A and B are set to 950 and 2050 respectively.
Then, C is set to 600

Checkpoints

- ❑ Redoing/undoing all transactions recorded in the log can be very slow
 1. Processing the entire log is time-consuming if the system has run for a long time
 2. We might unnecessarily redo transactions which have already output their updates to the database.

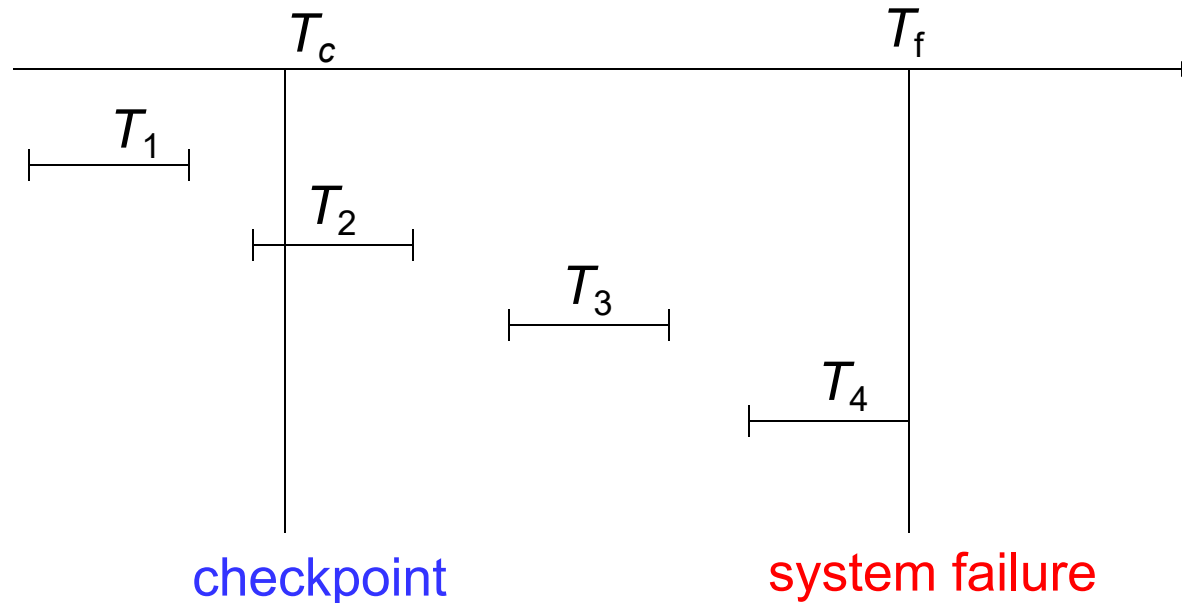
- ❑ Streamline recovery procedure by periodically performing **checkpointing**
 1. Output **all log records** currently residing in main memory onto stable storage.
 2. Output **all modified buffer blocks to the disk**.
 3. Write a log record **< checkpoint L >** onto stable storage where L is a list of all transactions active at the time of checkpoint.

➤ All updates are stopped while doing checkpointing

Checkpoints (Cont.)

- ❑ During recovery we need to consider only the most recent transaction T_i that started before the checkpoint, and transactions that started after T_i .
 - Scan backwards from end of log to find the most recent <checkpoint L > record
 - Only transactions that are in L or started after the checkpoint need to be redone or undone
 - Transactions that committed or aborted before the checkpoint already have all their updates output to stable storage.
- ❑ Some earlier part of the log may be needed for undo operations
 - Continue scanning backwards till a record < T_i start> is found for every transaction T_i in L .
 - Parts of log prior to earliest < T_i start> record above are not needed for recovery, and can be erased whenever desired.

Example of Checkpoints



- ❑ T_1 can be ignored (updates already output to disk due to checkpoint)
- ❑ T_2 and T_3 redone.
- ❑ T_4 undone

Outline

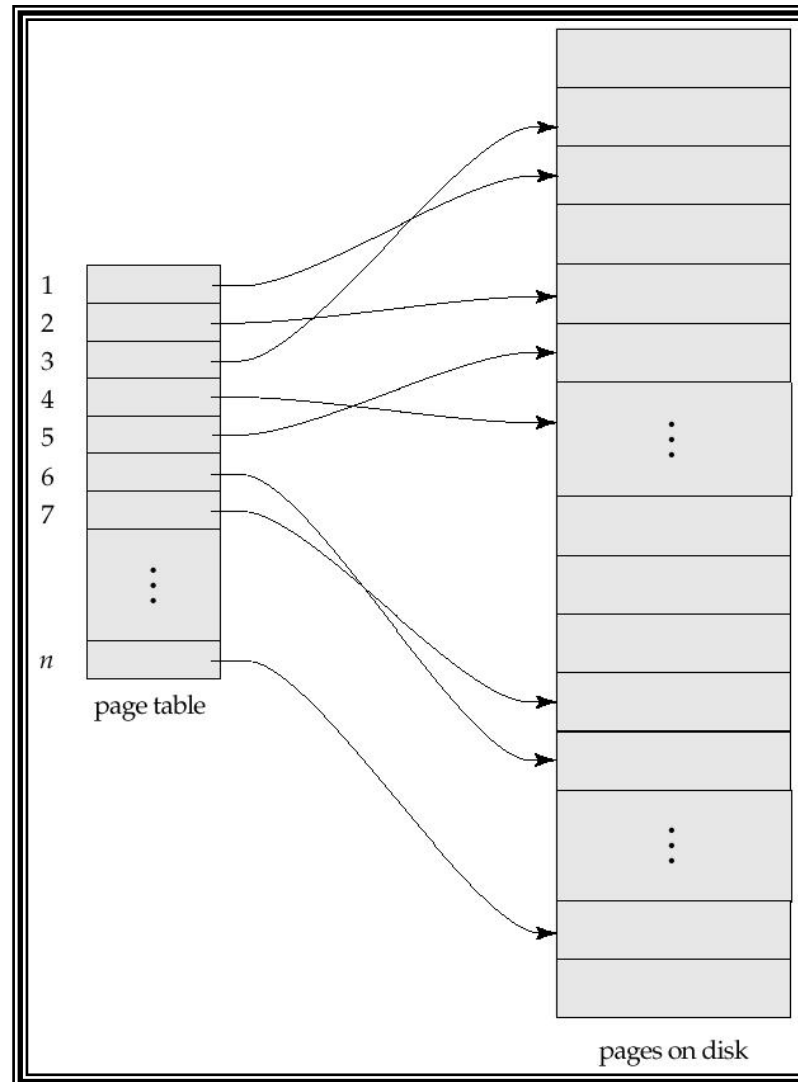
- ❑ Failure Classification
- ❑ *Storage Structure
- ❑ *Recovery and Atomicity
- ❑ Log-Based Recovery
- ❑ *Shadow Paging
- ❑ Recovery With Concurrent Transactions
- ❑ Buffer Management
- ❑ Failure with Loss of Nonvolatile Storage
- ❑ *Advanced Recovery Techniques
- ❑ *ARIES Recovery Algorithm
- ❑ *Remote Backup Systems



*Shadow Paging

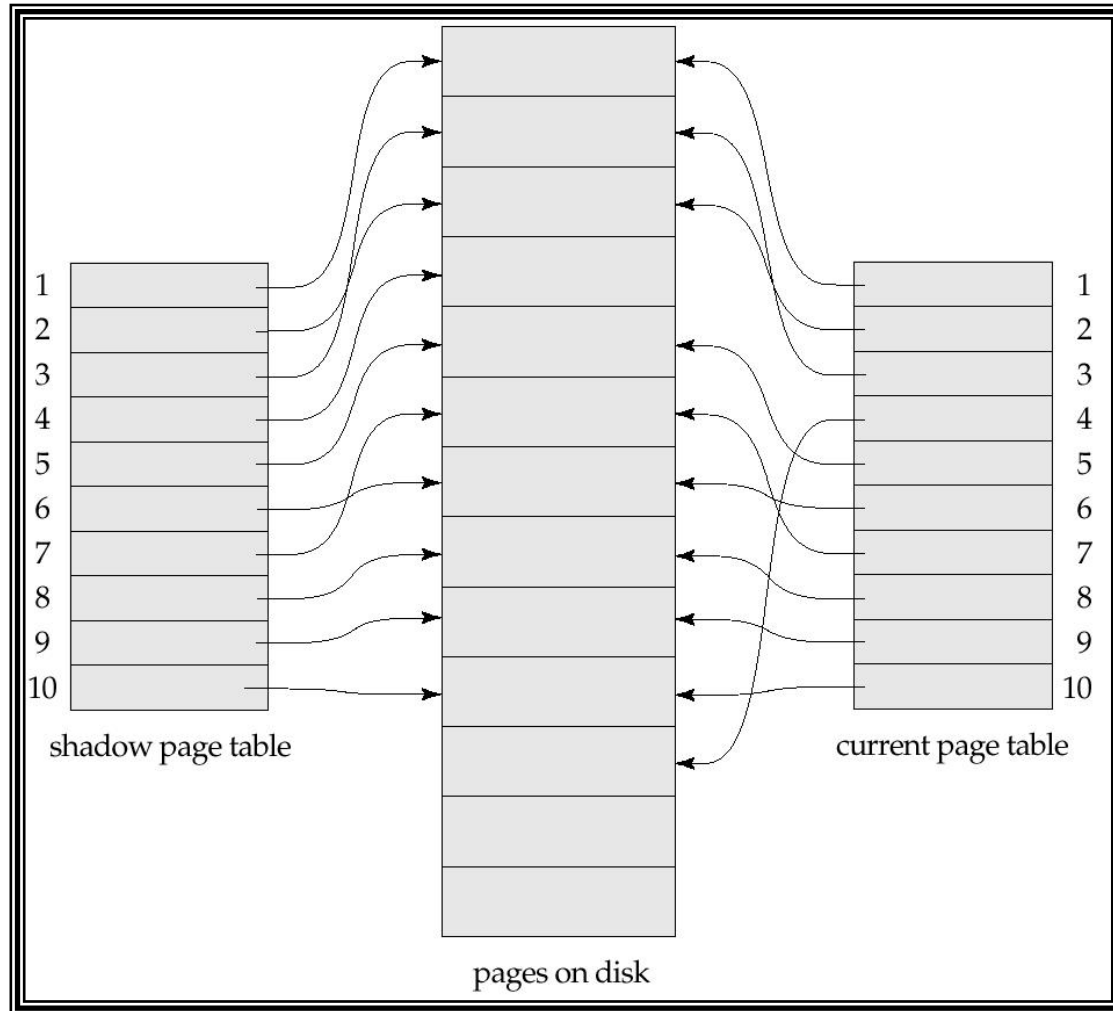
- ❑ **Shadow paging** is an alternative to log-based recovery; this scheme is useful if transactions execute serially
- ❑ **Idea:** maintain two page tables during the lifetime of a transaction –the **current page table**, and the **shadow page table**
- ❑ Store the shadow page table in nonvolatile storage, such that state of the database prior to transaction execution may be recovered.
 - Shadow page table is never modified during execution
- ❑ To start with, both the page tables are identical. Only current page table is used for data item accesses during execution of the transaction.
- ❑ Whenever any page is about to be written for the first time
 - A copy of this page is made onto an unused page.
 - The current page table is then made to point to the copy
 - The update is performed on the copy

Sample Page Table



Example of Shadow Paging

Shadow and current page tables after write to page 4



Shadow Paging (Cont.)

- ❑ To commit a transaction :
 1. Flush all modified pages in main memory to disk
 2. Output current page table to disk
 3. Make the current page table the new shadow page table, as follows:
 - Keep a pointer to the shadow page table at a fixed (known) location on disk.
 - To make the current page table the new shadow page table, simply update the pointer to point to current page table on disk
- ❑ Once pointer to shadow page table has been written, transaction is committed.
- ❑ No recovery is needed after a crash — new transactions can start right away, using the shadow page table.
- ❑ Pages not pointed to from current/shadow page table should be freed (garbage collected).

Shadow Paging (Cont.)

❑ Advantages of shadow-paging over log-based schemes

- No overhead of writing log records
- Recovery is trivial

❑ Disadvantages :

- Copying the entire page table is very expensive
 - Can be reduced by using a page table structured like a B⁺-tree
 - No need to copy entire tree, only need to copy paths in the tree that lead to updated leaf nodes
- Commit overhead is high even with above extension
 - Need to flush every updated page, and page table
- Data gets fragmented (related pages get separated on disk)
- After every transaction completion, the database pages containing old versions of modified data need to be garbage collected
- Hard to extend algorithm to allow transactions to run concurrently
 - Easier to extend log based schemes

Outline

- ❑ Failure Classification
- ❑ *Storage Structure
- ❑ *Recovery and Atomicity
- ❑ Log-Based Recovery
- ❑ *Shadow Paging
- ❑ Recovery With Concurrent Transactions
- ❑ Buffer Management
- ❑ Failure with Loss of Nonvolatile Storage
- ❑ *Advanced Recovery Techniques
- ❑ *ARIES Recovery Algorithm
- ❑ *Remote Backup Systems



Recovery With Concurrent Transactions

- ❑ We modify the log-based recovery schemes to allow multiple transactions to execute concurrently.
 - All transactions share a single disk buffer and a single log file
 - A buffer block can have data items updated by one or more transactions
- ❑ We assume concurrency control using strict two-phase locking; (-- 2PL with X-locks held until the end of the transaction.)
 - i.e., the updates of uncommitted transactions should not be visible to other transactions
- ❑ Logging is done as described earlier.
 - Log records of different transactions are interspersed in the log.
- ❑ The checkpointing technique and actions taken on recovery have to be changed
 - Since several transactions may be active when a checkpoint is performed.

Recovery With Concurrent Transactions (Cont.)

- ❑ Checkpoints are performed as before, except that the checkpoint log record is now of the form

< checkpoint L >

where L is the list of transactions active at the time of the checkpoint, like $\{ T_2, T_4 \}$

- We assume no updates are in progress while the checkpoint is carried out.

Recovery With Concurrent Transactions (Cont.)

❑ When the system recovers from a crash:

➤ ① It first does the following:

1. Initialize *undo-list* and *redo-list* to empty
2. Scan the log backwards from the end, **stopping scan** when the **first <checkpoint L > record** is found. Meanwhile, for each record found in log during the backward scan:
 1. If the record is $\langle T_i \text{ commit} \rangle$, add T_i to *redo-list*
 2. If the record is $\langle T_i \text{ start} \rangle$, then if T_i is not in *redo-list*, add T_i to *undo-list*
 3. If the record is $\langle T_i \text{ abort} \rangle$, add T_i to *undo-list*
3. For every T_i in L , if T_i is not in *redo-list*, add T_i to *undo-list*
 - At this point, *undo-list* consists of incomplete transactions which must be **undone**, and *redo-list* consists of finished transactions that must be **redone**.

Recovery With Concurrent Transactions (Cont.)

➤ ② Recovery now continues as follows:

1. Scan log backwards from the end of log file, stopping when $\langle T_i \text{ start} \rangle$ records have been encountered for every T_i in *undo-list*.
During the scan, perform undo for each log record that belongs to a transaction in *undo-list*.
2. Locate the most recent $\langle \text{checkpoint } L \rangle$ record.
3. Scan log forwards from the $\langle \text{checkpoint } L \rangle$ record till the end of the log.

During the scan, perform redo for each log record that belongs to a transaction on *redo-list*.

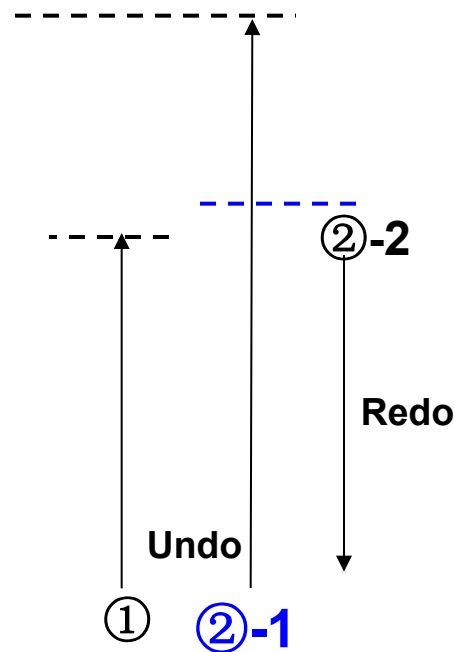
Example of Recovery

Go over the steps of the recovery algorithm on the following log:

~~<T₀ start>~~
 <checkpoint {T₀} >
 <T₀, A, 0, 10>
 <T₀ commit>
 <T₁ start>
 <T₁, A, 10, 15>
 <T₂ start>
 <T₂, C, 0, 10>
 <T₂, B, 10, 20>
 <checkpoint {T₁, T₂} >
 <T₃ start>
 <T₁ commit>
 <T₃, A, 15, 20>
 <T₃, D, 0, 10>
 <T₃ commit>
 - - -Crashed- - -

Undo-list:{ T₂}

Redo-list:{ T₁, T₃}



Results:
 A=20,
 B=10,
 C=0,
 D=10

Outline

- ❑ Failure Classification
- ❑ *Storage Structure
- ❑ *Recovery and Atomicity
- ❑ Log-Based Recovery
- ❑ *Shadow Paging
- ❑ Recovery With Concurrent Transactions
- ❑ **Buffer Management**
- ❑ Failure with Loss of Nonvolatile Storage
- ❑ *Advanced Recovery Techniques
- ❑ *ARIES Recovery Algorithm
- ❑ *Remote Backup Systems



Buffer Management ---- Log Record Buffering

- ❑ Typically, output to stable storage is in **unit of blocks**, and usually a log record is **much smaller than a block**.
- ❑ Therefore, **log records are buffered** in main memory, instead of being output directly to stable storage.
 - Log records are output to stable storage, when :
 - 1) Blocks of log records in the buffer **are full**,
 - 2) Or a log force operation is executed.
 - --- **E.g. checkpoint happens.**
- ❑ Log force (强制日志) is performed to commit a transaction by forcing all its log records (including the commit record $\langle T_i \text{ commit} \rangle$) to stable storage.
- ❑ Several log records can thus be output using a single output operation, reducing the I/O cost.

The Rules for Log Record Buffering

❑ The 4 rules below must be followed if log records are buffered:

- 1) Log records are output to stable storage in the order in which they are created.
- 2) Transaction T_i enters the commit state only when the log record $\langle T_i, \text{commit} \rangle$ has been output to stable storage.
- 3) Before the $\langle T_i, \text{commit} \rangle$ can be output to stable storage, all log records pertaining to T_i must have been output to stable storage.
- 4) Before a block of data in main memory is output to the database, all log records pertaining to the data in that block must have been output to stable storage. (日志应先于数据写到磁盘)
 - This rule is called the write-ahead logging rule (先写日志规则) or WAL
 - Strictly speaking WAL only requires undo information to be output

Database Buffering

- ❑ Database maintains an in-memory buffer of data blocks
 - When a new block is needed, if buffer is full an existing block needs to be removed from buffer
 - If the block chosen for removal has been updated, it must be output to disk
- ❑ The recovery algorithm supports the **no-force policy**, i.e., updated blocks need not be written to disk when transaction commits
 - **Force policy**: requires updated blocks to be written at commit
 - More expensive commit
- ❑ The recovery algorithm supports the **steal policy**, i.e., blocks containing updates of uncommitted transactions can be written to disk, even before the transaction commits

Database Buffering (Cont.)

- ❑ If a block with uncommitted updates is output to disk, log records with undo information for the updates are output to the log on stable storage first
 - (Write ahead logging)
- ❑ No updates should be in progress on a block when it is output to disk. Can be ensured as follows.
 - Before writing a data item, transaction acquires exclusive lock on block containing the data item
 - Lock can be released once the write is completed.
 - Such locks held for short duration are called **latches**.
- ❑ **To output a block to disk**
 1. First acquire an exclusive latch on the block
 1. Ensures no update can be in progress on the block
 2. Then perform a **log flush**
 3. Then output the block to disk
 4. Finally release the latch on the block

Buffer Management

- ❑ Database buffer can be implemented either
 - In an area of **real main-memory** reserved for the database, or
 - In **virtual memory**

- ❑ Implementing buffer **in reserved main-memory** has **drawbacks**:
 - **Memory is partitioned before-hand** between database buffer and applications, limiting flexibility.
 - **Needs may change**, and although operating system knows best how memory should be divided up at any time, **it cannot change the partitioning of memory**.

Buffer Management (Cont.)

❑ Database buffers are generally implemented in virtual memory in spite of some drawbacks:

- When operating system needs to evict a page that has been modified, the page is written to swap space on disk.
- When database decides to write buffer page to disk, buffer page may be in swap space, and may have to be read from swap space on disk and output to the database on disk, resulting in extra I/O!
 - Known as dual paging problem.
- Ideally when OS needs to evict a page from the buffer, it should pass control to database, which in turn should
 1. Output the page to database instead of to swap space (making sure to output log records first), if it is modified
 2. Release the page from the buffer, for the OS to useDual paging can thus be avoided, but common operating systems do not support such functionality.

Outline

- ❑ Failure Classification
- ❑ *Storage Structure
- ❑ *Recovery and Atomicity
- ❑ Log-Based Recovery
- ❑ *Shadow Paging
- ❑ Recovery With Concurrent Transactions
- ❑ Buffer Management
- ❑ **Failure with Loss of Nonvolatile Storage**
- ❑ *Advanced Recovery Techniques
- ❑ *ARIES Recovery Algorithm
- ❑ *Remote Backup Systems



Failure with Loss of Nonvolatile Storage

- ❑ So far we assumed no loss of non-volatile storage
- ❑ Technique similar to checkpointing used to deal with loss of non-volatile storage
 - Periodically **dump** the entire content of the database to stable storage
 - No transaction may be active during the dump procedure; a procedure similar to checkpointing must take place
 - Output all log records currently residing in main memory onto stable storage.
 - Output all buffer blocks onto the disk.
 - Copy the contents of the database to stable storage.
 - Output a record <**dump**> to log on stable storage.

Failure with Loss of Nonvolatile Storage (cont.)

- ❑ To **recover** from disk failure
 - Restore database from most recent dump.
 - Consult the log and redo all transactions that committed after the dump
- ❑ Can be extended to allow transactions to be active during dump; known as fuzzy dump or online dump
- ❑ Many security and reliability methods are used in various commercial DBMS.
- ❑ Various tools in commercial database products are used to help doing replica and recovery.

Outline

- ❑ Failure Classification
- ❑ *Storage Structure
- ❑ *Recovery and Atomicity
- ❑ Log-Based Recovery
- ❑ *Shadow Paging
- ❑ Recovery With Concurrent Transactions
- ❑ Buffer Management
- ❑ Failure with Loss of Nonvolatile Storage
- ❑ *Advanced Recovery Techniques
- ❑ ARIES Recovery Algorithm
- ❑ *Remote Backup Systems



*Advanced Recovery Techniques

- ❑ Support high-concurrency locking techniques, such as those used for B⁺-tree concurrency control
- ❑ Operations like B⁺-tree insertions and deletions release locks early.
 - They cannot be undone by restoring old values (physical undo), since once a lock is released, other transactions may have updated the B⁺-tree.
 - Instead, insertions (resp. deletions) are undone by executing a deletion (resp. insertion) operation (known as logical undo).
- ❑ For such operations, undo log records should contain the undo operation to be executed
 - Called logical undo logging, in contrast to physical undo logging.
- ❑ Redo information is logged physically (that is, new value for each write) even for such operations
 - Logical redo is very complicated since database state on disk may not be “operation consistent”

Advanced Recovery Techniques (Cont.)

❑ Operation logging is done as follows:

1. When operation starts, log $\langle T_i, O_j, \text{operation-begin} \rangle$. Here O_j is a unique identifier of the operation instance.
2. While operation is executing, normal log records with physical redo and physical undo information are logged.
3. When operation completes, $\langle T_i, O_j, \text{operation-end}, U \rangle$ is logged, where U contains information needed to perform a logical undo information.

❑ If crash/rollback occurs before operation completes:

- The operation-end log record is not found, and
- The physical undo information is used to undo operation.

❑ If crash/rollback occurs after the operation completes:

- The operation-end log record is found, and in this case
- Logical undo is performed using U ; the physical undo information for the operation is ignored.

❑ Redo of operation (after crash) still uses physical redo information.

Advanced Recovery Techniques (Cont.)

Rollback of transaction T_i is done as follows:

❑ Scan the log backwards

1. If a log record $\langle T_i, X, V_1, V_2 \rangle$ is found, perform the undo and log a special **redo-only log record** $\langle T_i, X, V_1 \rangle$.
2. If a $\langle T_i, O_j, \text{operation-end}, U \rangle$ record is found
 - Rollback the operation logically using the undo information U .
 - Updates performed during roll back are logged just like during normal operation execution.
 - At the end of the operation rollback, instead of logging an **operation-end** record, generate a record $\langle T_i, O_j, \text{operation-abort} \rangle$.
 - Skip all preceding log records for T_i until the record $\langle T_i, O_j, \text{operation-begin} \rangle$ is found

Advanced Recovery Techniques (Cont.)

- ❑ Scan the log backwards (cont.):
 3. If a redo-only record is found ignore it
 4. If a $\langle T_i, O_j, \text{operation-abort} \rangle$ record is found:
skip all preceding log records for T_i until the record $\langle T_i, O_j, \text{operation-begin} \rangle$ is found.
 5. Stop the scan when the record $\langle T_i, \text{start} \rangle$ is found
 6. Add a $\langle T_i, \text{abort} \rangle$ record to the log

Some points to note:

- ❑ Cases 3 and 4 above can occur only if the database crashes while a transaction is being rolled back.
- ❑ Skipping of log records as in case 4 is important to prevent multiple rollback of the same operation.

Advanced Recovery Techniques (Cont.)

The following actions are taken when recovering from system crash

1. Scan log forward from last < **checkpoint** L > record
 1. **Repeat history** by physically redoing all updates of all transactions,
 2. Create an undo-list during the scan as follows
 - *undo-list* is set to L initially
 - Whenever < T_i **start** > is found T_i is added to *undo-list*
 - Whenever < T_i **commit** > or < T_i **abort** > is found, T_i is deleted from *undo-list*

This brings database to state as of crash, with committed as well as uncommitted transactions having been redone.

Now undo-list contains transactions that are **incomplete**, that is, have neither committed nor been fully rolled back.

Advanced Recovery Techniques (Cont.)

Recovery from system crash (cont.)

2. Scan log backwards, performing undo on log records of transactions found in *undo-list*.
 - Transactions are rolled back as described earlier.
 - When $\langle T_i \text{ start} \rangle$ is found for a transaction T_i in *undo-list*, write a $\langle T_i \text{ abort} \rangle$ log record.
 - Stop scan when $\langle T_i \text{ start} \rangle$ records have been found for all T_i in *undo-list*
- This undoes the effects of incomplete transactions (those with neither **commit** nor **abort** log records). Recovery is now complete.

Advanced Recovery Techniques (Cont.)

❑ **Checkpointing** is done as follows:

1. Output all log records in memory to stable storage
2. Output to disk all modified buffer blocks
3. Output to log on stable storage a < **checkpoint** L> record.

Transactions are not allowed to perform any actions while checkpointing is in progress.

- ❑ Fuzzy checkpointing allows transactions to progress while the most time consuming parts of checkpointing are in progress
- Performed as described on next slide

Advanced Recovery Techniques (Cont.)

❑ **Fuzzy checkpointing** is done as follows:

1. Temporarily stop all updates by transactions
2. Write a <**checkpoint** L > log record and force log to stable storage
3. Note list M of modified buffer blocks
4. Now permit transactions to proceed with their actions
5. Output to disk all modified buffer blocks in list M
blocks should not be updated while being output
Follow WAL: all log records pertaining to a block must be output before the block is output
6. Store a pointer to the **checkpoint** record in a fixed position **last_checkpoint** on disk

❑ When recovering using a fuzzy checkpoint, start scan from the **checkpoint** record pointed to by **last_checkpoint**

- Log records before **last_checkpoint** have their updates reflected in database on disk, and need not be redone.
- Incomplete checkpoints, where system had crashed while performing checkpoint, are handled safely

Outline

- ❑ Failure Classification
- ❑ *Storage Structure
- ❑ *Recovery and Atomicity
- ❑ Log-Based Recovery
- ❑ *Shadow Paging
- ❑ Recovery With Concurrent Transactions
- ❑ Buffer Management
- ❑ Failure with Loss of Nonvolatile Storage
- ❑ *Advanced Recovery Techniques
- ❑ ***ARIES Recovery Algorithm**
- ❑ *Remote Backup Systems



*ARIES

❑ ARIES is a state of the art recovery method

- Incorporates numerous optimizations to reduce overheads during normal processing and to speed up recovery
- The recovery algorithm we studied earlier is modeled after ARIES, but greatly simplified by removing optimizations

❑ Unlike the recovery algorithm described earlier, ARIES

1. Uses **log sequence number (LSN)** to identify log records
 - Stores LSNs in pages to identify what updates have already been applied to a database page
2. Physiological redo
3. Dirty page table to avoid unnecessary redos during recovery
4. Fuzzy checkpointing that only records information about dirty pages, and does not require dirty pages to be written out at checkpoint time
 - More coming up on each of the above ...

ARIES Optimizations

❏ Physiological redo

- Affected page is physically identified, action within page can be logical
 - Used to reduce logging overheads
 - E.g., when a record is deleted and all other records have to be moved to fill hole
 - » Physiological redo can log just the record deletion
 - » Physical redo would require logging of old and new values for much of the page
 - Requires page to be output to disk atomically
 - Easy to achieve with hardware RAID, also supported by some disk systems
 - Incomplete page output can be detected by checksum techniques,
 - » But extra actions are required for recovery
 - » Treated as a media failure

ARIES Data Structures

❑ ARIES uses several data structures

- Log sequence number (LSN) identifies each log record
 - Must be sequentially increasing
 - Typically an offset from beginning of log file to allow fast access
 - Easily extended to handle multiple log files
- Page LSN
- Log records of several different types
- Dirty page table

ARIES Data Structures: Page LSN

- ❑ Each page contains a **PageLSN** which is the LSN of the last log record whose effects are reflected on the page
 - To update a page:
 - X-latch the page, and write the log record
 - Update the page
 - Record the LSN of the log record in PageLSN
 - Unlock page
 - To flush page to disk, must first S-latch page
 - Thus page state on disk is operation consistent
 - Required to support physiological redo
 - PageLSN is used during recovery to prevent repeated redo
 - Thus ensuring idempotence

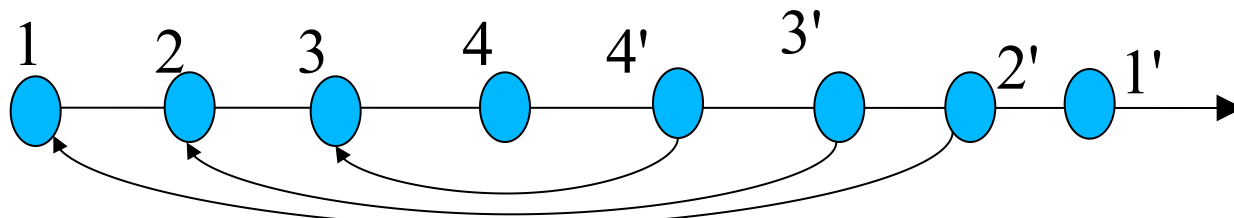
ARIES Data Structures: Log Record

- ❑ Each log record contains LSN of previous log record of the same transaction

LSN	TransID	PrevLSN	RedoInfo	UndoInfo
-----	---------	---------	----------	----------

- LSN in log record may be implicit
- ❑ Special redo-only log record called **compensation log record (CLR)** used to log actions taken during recovery that never need to be undone
 - Serves the role of operation-abort log records used in earlier recovery algorithm
 - Has a field UndoNextLSN to note next (earlier) record to be undone
 - Records in between would have already been undone
 - Required to avoid repeated undo of already undone actions

LSN	TransID	UndoNextLSN	RedoInfo
-----	---------	-------------	----------

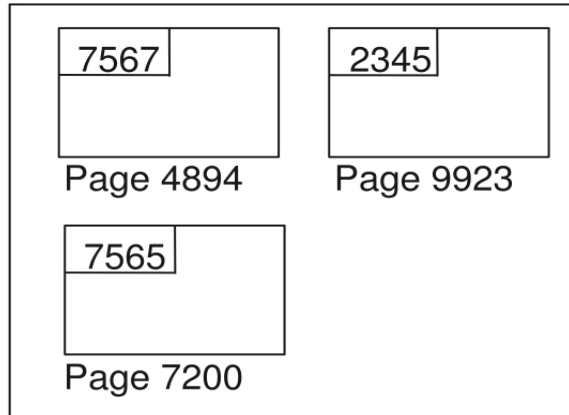


ARIES Data Structures: DirtyPage Table

❑ DirtyPageTable

- List of pages in the buffer that have been updated
- Contains, for each such page
 - **PageLSN** of the page
 - **RecLSN** is an LSN such that log records before this LSN have already been applied to the page version on disk
 - Set to current end of log when a page is inserted into dirty page table (just before being updated)
 - Recorded in checkpoints, helps to minimize redo work

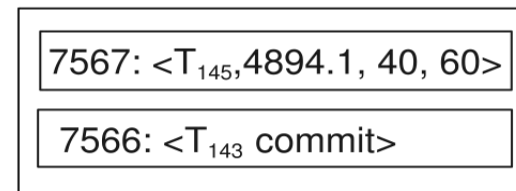
ARIES Data Structures



Database Buffer

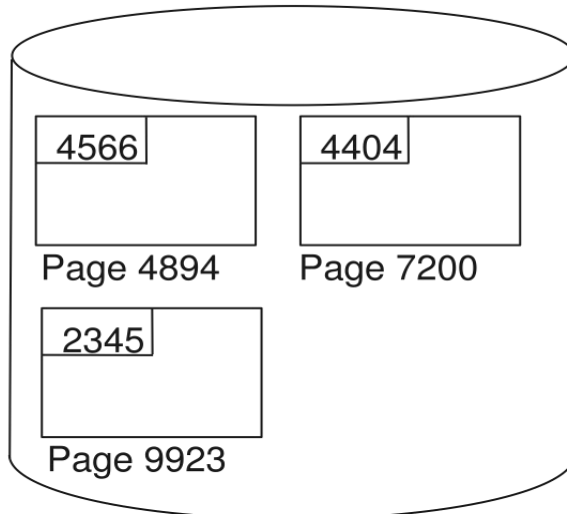
PageID	PageLSN	RecLSN
4894	7567	7564
7200	7565	7565

Dirty Page Table

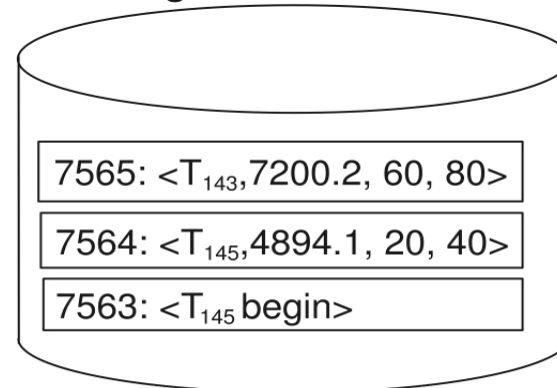


Log Buffer (PrevLSN and UndoNextLSN fields not shown)

Stable data



Stable log



ARIES Data Structures: Checkpoint Log

❑ Checkpoint log record

- Contains:
 - DirtyPageTable and list of active transactions
 - For each active transaction, LastLSN, the LSN of the last log record written by the transaction
- Fixed position on disk notes LSN of last completed checkpoint log record

❑ Dirty pages are not written out at checkpoint time

- Instead, they are flushed out continuously, in the background

❑ Checkpoint is thus very low overhead

- Can be done frequently

ARIES Recovery Algorithm

ARIES recovery involves three passes

❑ Analysis pass: Determines

- Which transactions to undo
- Which pages were dirty (disk version not up to date) at time of crash
- RedoLSN: LSN from which redo should start

❑ Redo pass:

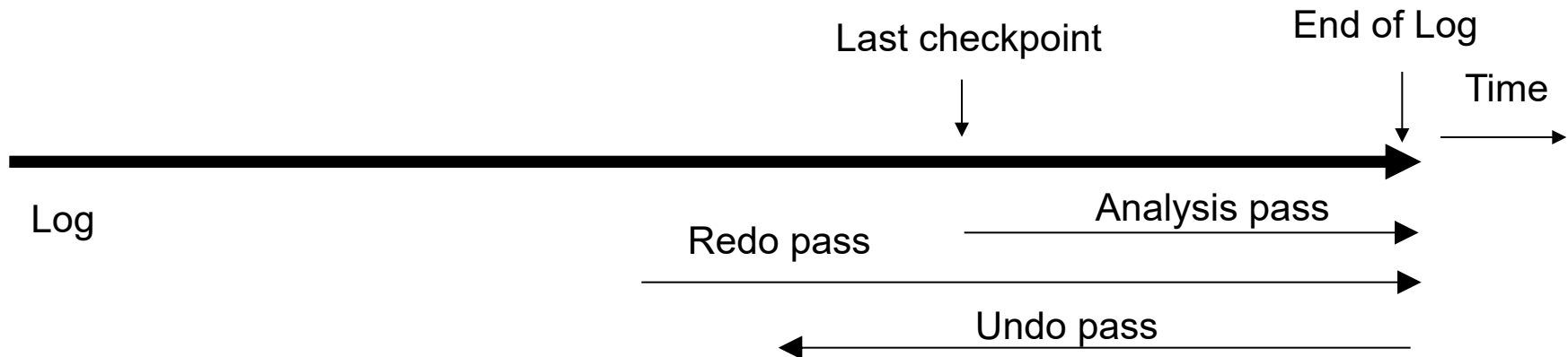
- Repeats history, redoing all actions from RedoLSN
 - RecLSN and PageLSNs are used to avoid redoing actions already reflected on page

❑ Undo pass:

- Rolls back all incomplete transactions
 - Transactions whose abort was complete earlier are not undone
 - Key idea: no need to undo these transactions: earlier undo actions were logged, and are redone as required

Aries Recovery: 3 Passes

- ❑ Analysis, redo and undo passes
- ❑ Analysis determines where redo should start
- ❑ Undo has to go back till start of earliest incomplete transaction



ARIES Recovery: Analysis

Analysis pass

- ❑ Starts from last complete checkpoint log record
 - Reads DirtyPageTable from log record
 - Sets RedoLSN = min of RecLSNs of all pages in DirtyPageTable
 - In case no pages are dirty, RedoLSN = checkpoint record's LSN
 - Sets undo-list = list of transactions in checkpoint log record
 - Reads LSN of last log record for each transaction in undo-list from checkpoint log record
- ❑ Scans forward from checkpoint
- ❑ .. Cont. on next page ...

ARIES Recovery: Analysis (Cont.)

Analysis pass (cont.)

❑ Scans forward from checkpoint

- If any log record found for transaction not in undo-list, adds transaction to undo-list
- Whenever an update log record is found
 - If page is not in DirtyPageTable, it is added with RecLSN set to LSN of the update log record
- If transaction end log record found, delete transaction from undo-list
- Keeps track of last log record for each transaction in undo-list
 - May be needed for later undo

❑ At end of analysis pass:

- RedoLSN determines where to start redo pass
- RecLSN for each page in DirtyPageTable used to minimize redo work
- All transactions in undo-list need to be rolled back

ARIES Redo Pass

Redo Pass: Repeats history by replaying every action not already reflected in the page on disk, as follows:

- ❑ Scans forward from RedoLSN. Whenever an update log record is found:
 1. If the page is not in DirtyPageTable or the LSN of the log record is less than the RecLSN of the page in DirtyPageTable, then skip the log record
 2. Otherwise fetch the page from disk. If the PageLSN of the page fetched from disk is less than the LSN of the log record, redo the log record

NOTE: if either test is negative the effects of the log record have already appeared on the page. First test avoids even fetching the page from disk!

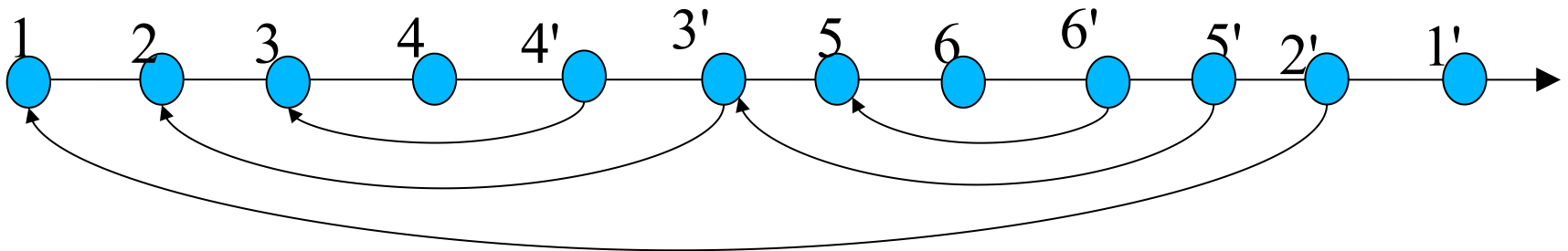
ARIES Undo Actions

□ When an undo is performed for an update log record

- Generate a CLR containing the undo action performed (actions performed during undo are logged physically or physiologically).
 - CLR for record n noted as n' in figure below
- Set UndoNextLSN of the CLR to the PrevLSN value of the update log record
 - Arrows indicate UndoNextLSN value

□ ARIES supports partial rollback

- Used e.g. to handle deadlocks by rolling back just enough to release reqd. locks
- Figure indicates forward actions after partial rollbacks
 - records 3 and 4 initially, later 5 and 6, then full rollback

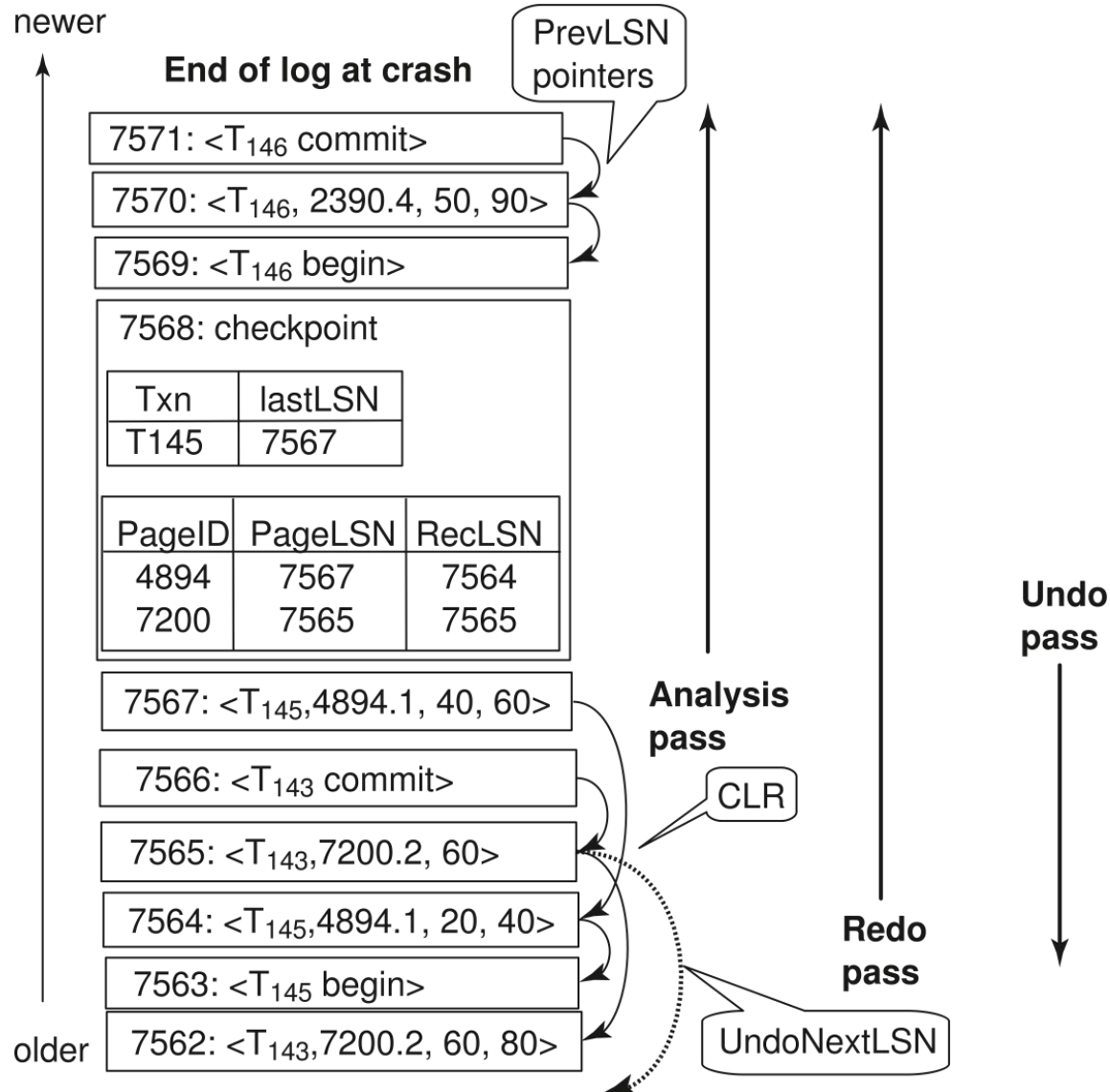


ARIES: Undo Pass

Undo pass:

- ❑ Performs backward scan on log undoing all transaction in undo-list
 - Backward scan optimized by skipping unneeded log records as follows:
 - Next LSN to be undone for each transaction set to LSN of last log record for transaction found by analysis pass.
 - At each step pick largest of these LSNs to undo, skip back to it and undo it
 - After undoing a log record
 - For ordinary log records, set next LSN to be undone for transaction to PrevLSN noted in the log record
 - For compensation log records (CLRs) set next LSN to be undo to UndoNextLSN noted in the log record
 - » All intervening records are skipped since they would have been undone already
- ❑ Undos performed as described earlier

Recovery Actions in ARIES



Other ARIES Features

❑ Recovery Independence

- Pages can be recovered independently of others
 - E.g., if some disk pages fail they can be recovered from a backup while other pages are being used

❑ Savepoints:

- Transactions can record savepoints and roll back to a savepoint
 - Useful for complex transactions
 - Also used to rollback just enough to release locks on deadlock

Other ARIES Features (Cont.)

❑ Fine-grained locking:

- Index concurrency algorithms that permit tuple level locking on indices can be used
 - These require logical undo, rather than physical undo, as in earlier recovery algorithm

❑ Recovery optimizations: For example:

- Dirty page table can be used to **prefetch** pages during redo
- Out of order redo is possible:
 - redo can be postponed on a page being fetched from disk, and performed when page is fetched.
 - Meanwhile other log records can continue to be processed

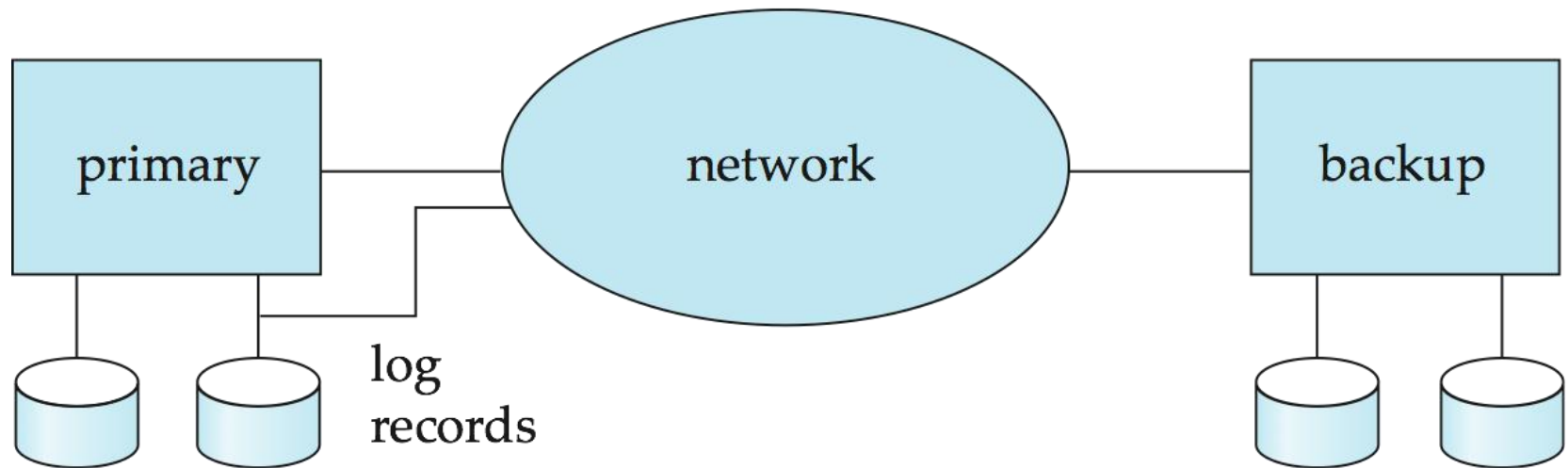
Outline

- ❑ Failure Classification
- ❑ *Storage Structure
- ❑ *Recovery and Atomicity
- ❑ Log-Based Recovery
- ❑ *Shadow Paging
- ❑ Recovery With Concurrent Transactions
- ❑ Buffer Management
- ❑ Failure with Loss of Nonvolatile Storage
- ❑ *Advanced Recovery Techniques
- ❑ *ARIES Recovery Algorithm
- ❑ *Remote Backup Systems



*Remote Backup Systems

- ❑ Remote backup systems provide high availability by allowing transaction processing to continue even if the primary site is destroyed.



Remote Backup Systems (Cont.)

❑ **Detection of failure:** Backup site must detect when primary site has failed

- To distinguish primary site failure from link failure maintain several communication links between the primary and the remote backup.
- Heart-beat messages

❑ **Transfer of control:**

- To take over control backup site first perform recovery using its copy of the database and all the log records it has received from the primary.
 - Thus, completed transactions are redone and incomplete transactions are rolled back.
- When the backup site takes over processing it becomes the new primary
- To transfer control back to old primary when it recovers, old primary must receive redo logs from the old backup and apply all updates locally.

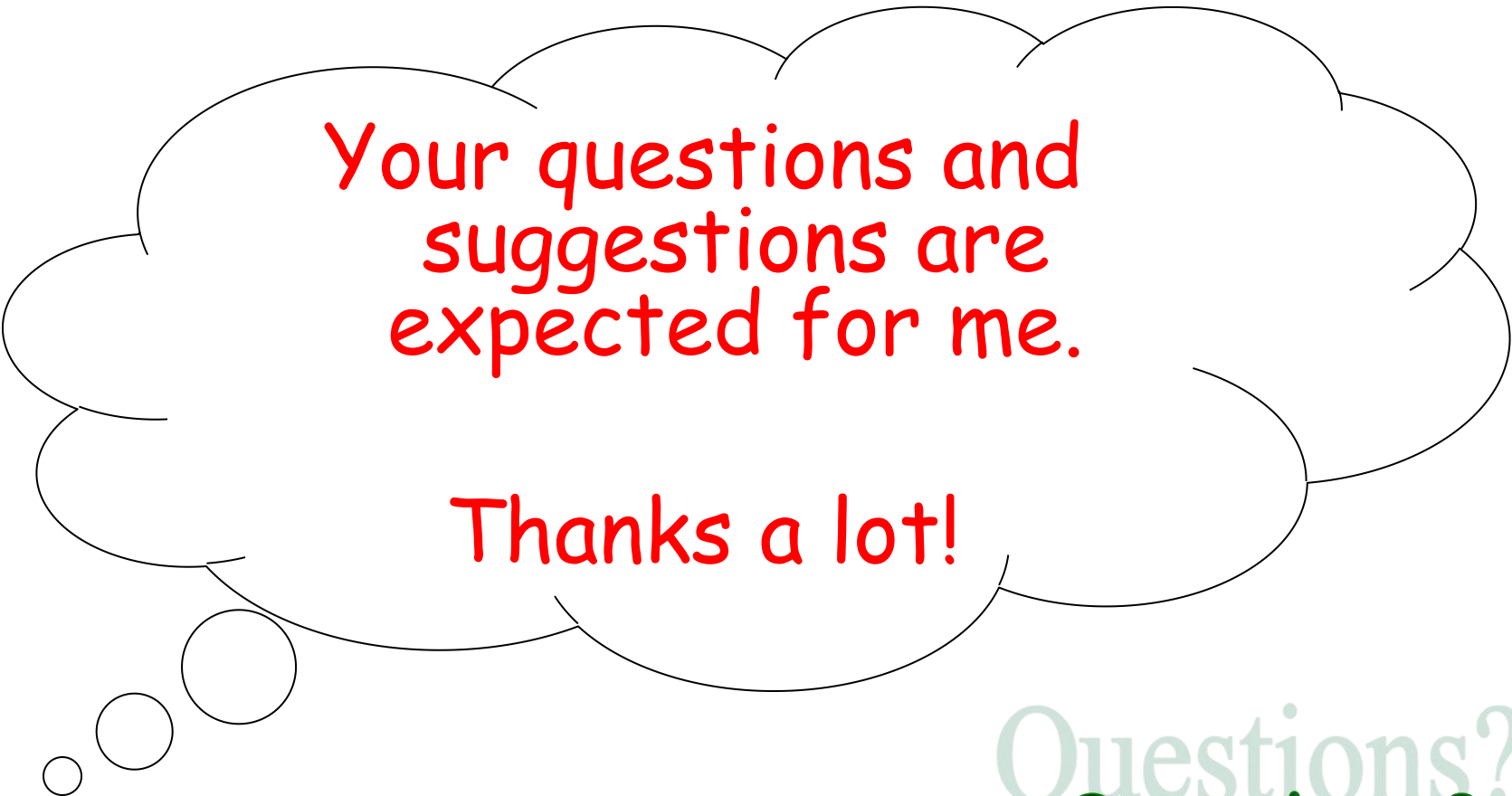
Remote Backup Systems (Cont.)

- ❑ **Time to recover:** To reduce delay in takeover, backup site periodically processes the redo log records (in effect, performing recovery from previous database state), performs a checkpoint, and can then delete earlier parts of the log.
- ❑ **Hot-Spare** configuration permits very fast takeover:
 - Backup continually processes redo log record as they arrive, applying the updates locally.
 - When failure of the primary is detected the backup rolls back incomplete transactions, and is ready to process new transactions.
- ❑ Alternative to remote backup: distributed database with replicated data
 - Remote backup is faster and cheaper, but less tolerant to failure
 - more on this in Chapter 19

Remote Backup Systems (Cont.)

- ❑ Ensure durability of updates by delaying transaction commit until update is logged at backup; avoid this delay by permitting lower degrees of durability.
- ❑ **One-safe**: commit as soon as transaction's commit log record is written at primary
 - Problem: updates may not arrive at backup before it takes over.
- ❑ **Two-very-safe**: commit when transaction's commit log record is written at primary and backup
 - Reduces availability since transactions cannot commit if either site fails.
- ❑ **Two-safe**: proceed as in two-very-safe if both primary and backup are active. If only the primary is active, the transaction commits as soon as its commit log record is written at the primary.
 - Better availability than two-very-safe; avoids problem of lost transactions in one-safe.

Q & A



Your questions and
suggestions are
expected for me.

Thanks a lot!

Exercises: 19.1, 19.2, and 19.22 (see
Pages 952-955)

Questions?
Questions?